TEMCO LEARN

T3 Firmware & Bootloader

Inside the bootloader · built Nov 13, 2026

Inside the bootloader

The variables, the checksum verification and the handler that receives a write, plus the files worth reading and the errors the flow can end in.

9. Code Implementation Details

Key Variables (app_main.c)

C

```c
// Firmware Update State
uint8_t com_update_firmware = 0;           // Update in progress flag
uint8_t com_update_firmware_timer = 0;     // Watchdog counter
uint8_t update_flag = 0;                   // 0x55 when successful
uint16_t rs485_resume_chunk = 0;           // Packet count for resume

// OTA Handle & Partition
esp_ota_handle_t update_com_handle = 0;
const esp_partition_t *update_com_partition = NULL;

// Buffering
#define OTA_PACKET_SIZE 128u
static uint8_t s_rs485_ota_chunk_buf[OTA_FLASH_CHUNK_SIZE];  // 4096 bytes
static size_t s_rs485_ota_chunk_len = 0;   // Current buffer fill
static uint32_t s_rs485_ota_next_offset = 0;  // Flash write offset

// Resume Tracking
uint8_t com_flash_high = 0;                // Packet count HIGH byte
uint8_t com_flash_low = 0;                 // Packet count LOW byte
uint32_t rs485_resume_bytes = 0;           // Backup variable
uint8_t rs485_run_begin_one_time = 0;      // One-time esp_ota_begin flag
```

MD5 Verification

```c
// MD5 Registers (1995-1998) store expected firmware checksum
unsigned short firmware_md5[4];

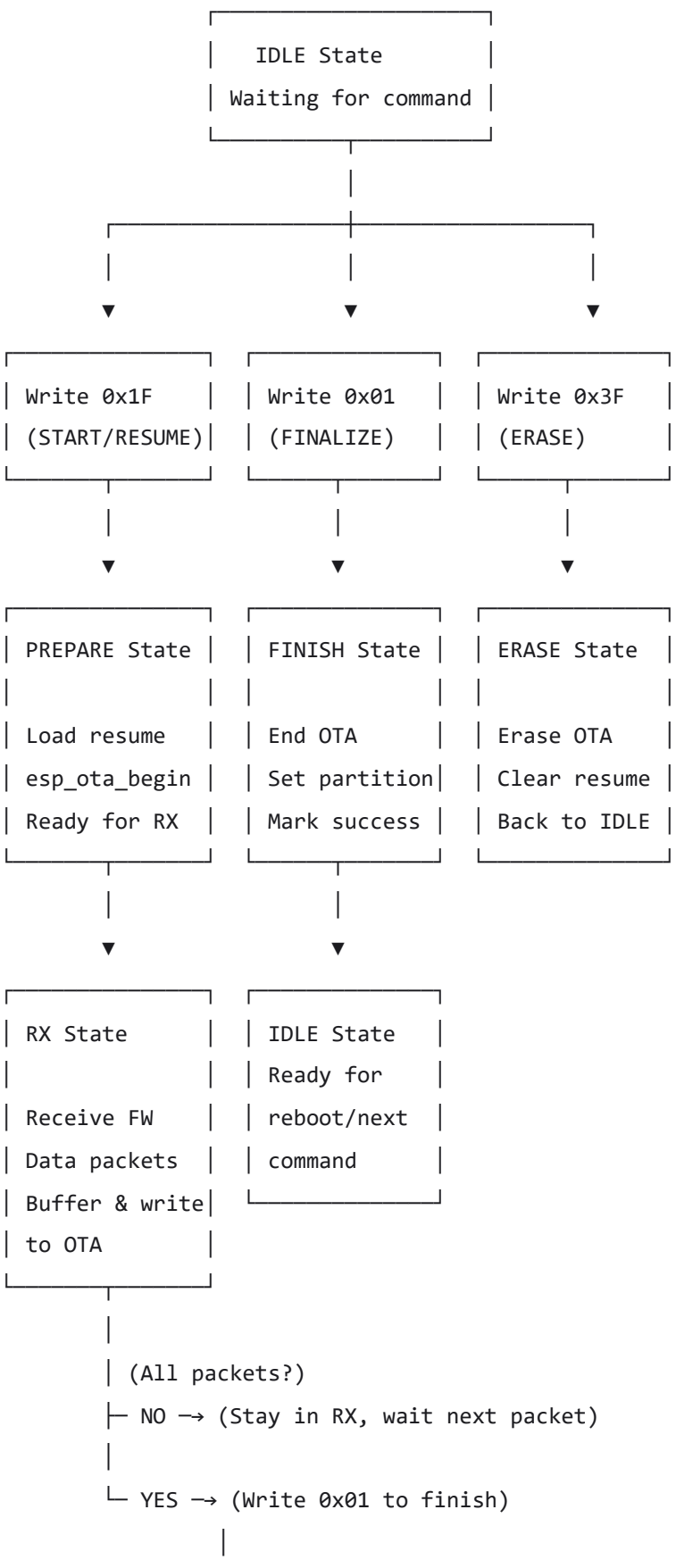
// On completion, device can compare:
// - Hardware computed MD5 from esp_ota_end()
// - vs. ISP tool provided MD5 (in registers 1995-1998)
```

Modbus Write Handler (Partial Code)

```c
if (address == UPDATE_STATUS) {
    if (*(bufadd+5) == 0x1F) {
        // START/RESUME Update
        com_update_firmware = 1;
        update_com_partition = esp_ota_get_next_update_partition(NULL);
        update_ota_address(rs485_resume_chunk);  // Derive resume offset
        esp_ota_begin(update_com_partition, OTA_SIZE_UNKNOWN, &update_com_handle);
        s_rs485_ota_next_offset = rs485_resume_chunk * OTA_PACKET_SIZE;
        rs485_run_begin_one_time = 1;
    }
    else if (*(bufadd+5) == 0x01) {
        // FINISH Update
        esp_ota_end(update_com_handle);
        esp_ota_set_boot_partition(update_com_partition);
        update_flag = 0x55;  // Success marker
        rs485_resume_chunk = 0;
        save_uint32_to_flash(FLASH_RS_RES_PACKAGE, 0);  // Clear resume
        com_update_firmware = 0;
    }
    else if (*(bufadd+5) == 0x3F) {
        // ERASE OTA Partition
        esp_partition_erase_range(update_com_partition, 0, update_com_partition->size);
        rs485_resume_chunk = 0;
        save_uint32_to_flash(FLASH_RS_RES_PACKAGE, 0);
    }
}
```

10. Flowchart - State Machine Overview

TEXT



```
                    ▼
            (Goto FINISH State above)
```

---

11. Error Handling

TEXT

```
Error Scenarios:

1. Invalid Partition:
   ├─ esp_ota_get_next_update_partition() returns NULL
   ├─ Log: "No OTA partition available"
   └─ Action: Return error, stay in IDLE

2. OTA Begin Failure:
   ├─ esp_ota_begin() returns error
   ├─ Log: "esp_ota_begin failed"
   └─ Action: esp_restart() to recover

3. Flash Write Error:
   ├─ esp_ota_write_with_offset() fails
   ├─ Log: "OTA flash write failed at offset X"
   └─ Action: Return error, but resume position saved

4. Image Validation Failure:
   ├─ esp_ota_end() returns ESP_ERR_OTA_VALIDATE_FAILED
   ├─ Log: "Image validation failed, image is corrupted"
   └─ Action: Do NOT call esp_ota_set_boot_partition()
   └─ Recovery: Write 0x3F to erase and retry

5. Boot Partition Switch Failure:
   ├─ esp_ota_set_boot_partition() returns error
   ├─ Log: "esp_ota_set_boot_partition failed"
   └─ Action: Log error but DO NOT restart (allows retry)
```

---

13. Key Files Reference

- **app_main.c** - Main update logic & Modbus handlers (lines 800-1000+)
- **temco_dfu.c** - DFU protocol implementation (if using TFTP)

- **flash.c** - NVS storage for resume state
- **modbus.h** - Register address definitions
- **temco_partitions.csv** - Partition layout definition
- **FIRMWARE_UPDATE_PROCESS.md** - Original detailed process docs

---

14. Summary

Aspect	Details
OTA Partition Address	0x120000 (6 MB)
OTA Select Data	0x19000 (2 KB)
Packet Size	128 bytes
Resume Granularity	1 packet (128 bytes)
Max Packets	~48,000 (6 MB ÷ 128 bytes)
Update Commands	0x1F (start/resume), 0x01 (finish), 0x3F (erase)
MD5 Registers	1995-1998
Success Flag	UPDATE_FLAG = 0x55
Recovery Method	Automatic resume on power loss